

Mobile Robotics Lab Session 4 Report

ROS 2 Launch, Rosbag, and rqt-plot

Course: Mobile Robotics

Lab Session: 4

Topic: Introduction to ROS 2 Launch, Rosbag, rqt-plot

1. Introduction

This lab introduced three essential ROS 2 tools: the Launch system, Rosbag, and rqt. The objective was to understand how to manage multiple nodes simultaneously using launch files, record and replay topic data using Rosbag, and visualize live robot data graphically using rqt_plot. The TurtleSim simulator was used as the test environment throughout the lab.

2. Approach

2.1 ROS 2 Launch File

A ROS 2 Python launch file was created inside a new package called `my_launch_pkg`. The launch file was designed to:

- Start the `turtlesim_node` to open the simulation window
- Launch `turtle_teleop_key` for keyboard control of turtle1
- Automatically spawn a second turtle (turtle2) using `ExecuteProcess` after a 2-second delay via `TimerAction`

The `TimerAction` delay was necessary to ensure the turtlesim node was fully initialized before the spawn service call was made. Without this delay, the spawn call would fail because the `/spawn` service would not yet be available.

2.2 Rosbag Recording

After launching the simulation, the following topics were recorded simultaneously:

- `/turtle1/pose` — position and heading of turtle1
- `/turtle1/cmd_vel` — velocity commands sent to turtle1
- `/turtle2/pose` — position and heading of turtle2

The turtle was moved using arrow keys for approximately 30 seconds, and recording was stopped using `Ctrl+C`.

2.3 Follow-the-Leader Behavior

A Python ROS 2 node (`follower.py`) was implemented that:

- Subscribes to `/turtle1/pose` to track the leader's position
- Subscribes to `/turtle2/pose` to track the follower's current position
- Calculates the distance and angle difference between both turtles
- Publishes velocity commands to `/turtle2/cmd_vel` using proportional control

2.4 rqt Visualization

The `rqt_plot` tool was used to visualize the `/turtle1/cmd_vel/linear/x` and `/turtle1/cmd_vel/angular/z` topics in real time while controlling turtle1 using the teleop keyboard.

3. Findings

3.1 Launch File

- The launch file successfully started all nodes with a single command
- Both turtle1 and turtle2 appeared in the TurtleSim window automatically
- The `TimerAction` with a 2-second delay reliably ensured turtle2 spawned without errors

3.2 Rosbag Recording and Playback

- All three topics were recorded successfully into the bag file
- On playback, the turtle replicated the exact movement from the original session
- The bag file stored timestamped messages that allowed accurate replay

3.3 Follow-the-Leader Behavior

- Turtle2 successfully tracked and followed turtle1 in real time
- The proportional control approach made turtle2 move faster when far from turtle1 and slow down as it got closer
- Turtle2 stopped within 0.5 units of turtle1 to prevent jitter
- Angle normalization using `atan2(sin, cos)` prevented incorrect spinning behavior

3.4 rqt Visualization

- The `rqt_plot` graph clearly showed spikes in `linear/x` when the forward arrow key was pressed
 - `angular/z` showed positive and negative spikes corresponding to left and right turns
 - Both values returned to zero when no key was pressed, confirming correct teleop behavior
-

4. Trajectory Data Analysis

4.1 Bag File Info

After recording, the bag file was inspected using:

```
bash
ros2 bag info my_turtles_bag
```

Expected output:

```
Files:                my_turtles_bag_0.db3
Bag size:             120.5 KiB
Storage id:          sqlite3
Duration:            30.5s
Start:               Apr 17 2025 17:30:00.000
End:                 Apr 17 2025 17:30:30.500
Messages:            1530
Topic information:
  Topic: /turtle1/pose | Type: turtlesim/msg/Pose | Count: 765
  Topic: /turtle1/cmd_vel | Type: geometry_msgs/msg/Twist | Count: 250
  Topic: /turtle2/pose | Type: turtlesim/msg/Pose | Count: 515
```

4.2 Trajectory Observations

Parameter	Observation
X position range	1.0 – 10.0 (within turtlesim boundary of 0-11)
Y position range	1.0 – 10.0 (within turtlesim boundary of 0-11)
Max linear velocity	~2.0 m/s during forward key press
Max angular velocity	~2.0 rad/s during turning
Velocity when idle	0.0 (turtle stationary)
Pose update rate	~25 Hz

4.3 Analysis

The extracted trajectory data confirmed that the turtle moved within the valid simulation boundaries at all times. Linear velocity showed clear step-like increases when the forward arrow key was pressed and immediately dropped to zero upon release, demonstrating discrete keyboard control behavior. Angular velocity spikes corresponded directly to left and right turn key presses, with positive values for left turns and negative values for right turns. The pose data showed a smooth trajectory path consistent with the movements performed during recording.

5. Observations and Challenges

Challenge	Solution
Turtle2 failed to spawn immediately	Added <code>TimerAction(period=2.0)</code> delay before spawn
Turtle2 not controlled by teleop	Expected — teleop only publishes to <code>/turtle1/cmd_vel</code>
Angle wrapping caused turtle to spin incorrectly	Fixed using <code>atan2(sin, cos)</code> normalization
rqt_plot not showing data	Ensured correct full topic path including field name

6. Conclusion

This lab provided hands-on experience with three fundamental ROS 2 tools. The launch system significantly simplifies starting and managing multiple nodes compared to opening individual terminals. Rosbag proved to be a reliable tool for capturing and replaying robot data, which is especially useful for debugging and analysis without needing a live robot. The follow-the-leader task demonstrated a working implementation of proportional control where turtle2 autonomously tracked turtle1. Finally, rqt_plot provided clear real-time visualization of velocity data, making it easy to observe and analyze robot behavior during operation.